

Universidad de Antioquia

Facultad de Ingeniería

Informática II — Semestre 2026-1

Implementación de Tetris mediante manipulación de bits en C++

Autores:

Gustavo Valencia Mercado
Pablo Gonzalez Rojas

Profesor:

Aníbal Guerra / Augusto Salazar

Medellín, 22 de marzo de 2026

Índice

1	Análisis del problema y consideraciones para la solución	2
2	Esquema de tareas y arquitectura del sistema	3
3	Algoritmos implementados	3
3.1	Representación del tablero	3
3.2	Representación de piezas	4
3.3	Detección de colisiones	5
3.4	Rotación de piezas	6
3.5	Fijación de piezas	8
3.6	Limpieza de filas	9
4	Problemas de desarrollo	10
5	Evolución de la solución	10

1. Análisis del problema y consideraciones para la solución

El desafío número 1 del curso, propone el desarrollo de un motor de Tetris, mostrado de manera estática en la consola del entorno QT, utilizando memoria dinámica, punteros y manipulación de bits. Los requerimientos para la lógica del juego siguen los lineamientos del juego original, es decir, las piezas clásicas y sus respectivas rotaciones, así como las mismas reglas para el transcurso del juego, con la diferencia de que en este caso, sería implementado como un juego por turnos y con una visualización estática del tablero. Asimismo, se establecen distintas restricciones para el desarrollo del juego, tales como el uso de STL, structs, strings y cualquier librería que resuelva directamente el problema, esto con el fin de fomentar el pensamiento crítico y lograr un mejor desarrollo de la lógica de programación. Para representar el tablero, la base de todo el juego, optamos por una matriz dinámica orquestada por punteros dobles de tipo `uint8_t` cuyas dimensiones serían escogidas por el usuario bajo ciertas restricciones, donde cada fila es un arreglo independiente de bytes y cada bit representa una celda de esta matriz. En cuanto a las piezas, se utilizaron arreglos dinámicos de 3 `uint8_t`, de los cuales se usaron únicamente los 3 bits más significativos(7,6,5) para la construcción de la pieza. Para todo lo mencionado anteriormente, se utilizó el tipo `uint8_t`, ya que esto erradica el problema de *endianness*, lo cual se refiere a la forma en que cada máquina llega a representar conjuntos de más de un byte, lo cual podría ser un problema para la ejecución del juego, ya que dependiendo del esquema utilizado por la máquina (*Little -endian* o *Big-endian*), esto podría invertir todo el funcionamiento del juego. Con uso de `uint8_t` se trabaja byte a byte, evitando caer en el conflicto mencionado. Cabe aclarar que para las operaciones de alineación de bits al colocar piezas en el tablero, se utiliza internamente un `uint32_t` como espacio de trabajo temporal, lo que permite desplazar los bits de la pieza sin perder información al cruzar la frontera entre bytes. Este valor nunca se escribe directamente a memoria, sino que se descompone en bytes individuales mediante máscaras `& 0xFF`. Por esto, resulta necesario que el ancho seleccionado para el tablero sea múltiplo de 8, ya que de esta manera se podrá representar cada bloque de 8 columnas(bits) con 1 byte, facilitando la lógica. Por último, para la representación de las piezas, se optó por utilizar los bits más significativos para la parte activa de la matriz (la que da forma a la pieza), puesto que de este modo se simplifican las operaciones de desplazamiento. Para conseguir un desarrollo óptimo de todo lo planteado, se dividió el desafío en 4 módulos, de forma que se pudiera trabajar las partes de manera aislada y organizada, estos fueron:

- **board**, validación, impresión y limpieza del tablero.
- **pieces**, generación y rotaciones de las fichas.
- **game**, lógica de juego (colisiones, desplazamientos, fijación de fichas, game over).
- **main**, flujo general del juego.

2. Esquema de tareas y arquitectura del sistema

El proyecto se estructuró en 4 módulos independientes, cada uno con un par de archivos `.h` y `.cpp`. Los archivos `.h` contienen el prototipado de las funciones y el `.cpp` su implementación. Esta separación permite compilar y depurar cada módulo de forma aislada.

Módulo	Responsabilidad	Funciones principales
<code>board</code>	Gestión del tablero	<code>crear_tablero</code> , <code>liberar_tablero</code> , <code>imprimir_tablero</code> , <code>limpiarFilas</code>
<code>pieces</code>	Gestión de piezas	<code>generarPieza</code> , <code>rotacion</code> , <code>liberarPieza</code>
<code>game</code>	Lógica del juego	<code>hayColision</code> , <code>moverIzquierda</code> , <code>moverDerecha</code> , <code>bajar</code> , <code>fijarPieza</code> , <code>hayGameOver</code>
<code>main</code>	Flujo general	Bucle principal, entrada del usuario

Cuadro 1: Módulos del proyecto y sus funciones principales

Las dependencias entre módulos siguen una jerarquía clara. Los módulos **board**, **game** y **pieces** son independientes entre sí, mientras que el **main** depende de los tres módulos restantes para orquestar el flujo completo:

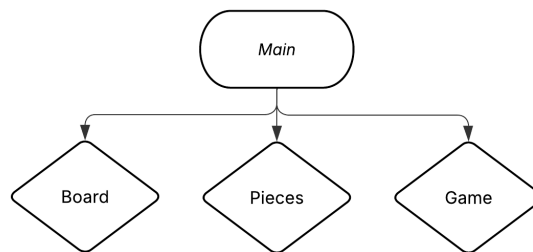


Figura 1: Jerarquía de los módulos

3. Algoritmos implementados

3.1. Representación del tablero

La representación del tablero está dada de forma dinámica por punteros dobles de `ancho/8 bytes` (`uint8_t`) que apuntan cada uno a una fila independiente, donde cada bit representa una celda del tablero, la cual estará representada por `#` (1) si está ocupada y `.`(0) si el espacio está libre. Dicha estructura de datos resulta más eficiente, ya que la memoria correspondiente a cada fila eliminada se libera y simplemente se mueven los

punteros, es decir, se trabaja no directamente con el contenido sino con el lugar al que se apunta.

```
1 crear_tablero(alto, ancho):
2     bytesPerFila = ancho / 8
3     tablero = nuevo arreglo de (alto) punteros
4     para i = 0 hasta alto-1:
5         tablero[i] = nuevo arreglo de (bytesPerFila) bytes,
6                     inicializado en 0
7     retornar tablero
```

Listing 1: Pseudocódigo – Creación del tablero

```
1 uint8_t** crear_tablero(int alto, int ancho) {
2     int bytesFila = ancho / 8;
3     uint8_t** tablero = new uint8_t*[alto];
4     for(int i = 0; i < alto; i++) {
5         tablero[i] = new uint8_t[bytesFila]();
6     }
7     return tablero;
8 }
```

Listing 2: Implementación – Creación del tablero

3.2. Representación de piezas

Para representar las piezas, optamos por matrices 3x3, utilizando como base un arreglo dinámico de 3 `uint8_t`, cuyos bits más significativos(7,6,5) serán los encargados de dar forma a la pieza deseada. El arreglo siempre tiene 3 filas, incluso cuando la pieza solo ocupa 2, porque al rotar una pieza que ocupa 2 filas puede pasar a ocupar 3, y reducir el tamaño del arreglo causaría acceso a memoria inválida.

```
1 generarPieza(filas, tipo):
2     tipo = numero aleatorio entre 0 y 5
3     pieza = nuevo arreglo de 3 bytes, inicializado en 0
4     segun tipo:
5         caso 0: pieza[0]=11000000, pieza[1]=11000000
6         caso T: pieza[0]=11100000, pieza[1]=01000000
7         caso S: pieza[0]=01100000, pieza[1]=11000000
8         caso Z: pieza[0]=11000000, pieza[1]=01100000
9         caso J: pieza[0]=10000000, pieza[1]=11100000
10        caso L: pieza[0]=00100000, pieza[1]=11100000
11        pieza[2] = 00000000 (siempre)
12    filas = 3
13    retornar pieza
```

Listing 3: Pseudocódigo – Generación de pieza

```

1 uint8_t* generarPieza(int &filas, int &tipo) {
2     int num = rand() % 6;
3     tipo = num;
4     uint8_t* pieza = new uint8_t[3]();
5     filas = 3;
6     switch(tipo) {
7     case 0: // 0
8         pieza[0] = 0b11000000;
9         pieza[1] = 0b11000000;
10        break;
11    case 1: // T
12        pieza[0] = 0b11100000;
13        pieza[1] = 0b01000000;
14        break;
15    // ... resto de casos
16    }
17    return pieza;
18 }

```

Listing 4: Implementación – Generación de pieza (fragmento)

3.3. Detección de colisiones

Para detectar las colisiones entre bits, primero se verifican dinámicamente los bordes en X y en Y de las piezas, a partir del bit más significativo de la pieza en ese momento(lo que evita problemas con las rotaciones) y comparándola con las fronteras del tablero. Luego, para cada fila activa de la pieza, se expande el byte a **uint32_t** y se desplaza piezaX 24 posiciones a la derecha para alinear los bits al tablero, con esto, evitamos la pérdida de bits al saltar de un byte a otro. Finalmente se extrae byte a byte y se hace AND contra el tablero, si el resultado es distinto de 0, hay superposición y por lo tanto colisión.

```

1 hayColision(tablero, alto, ancho, pieza, piezaX, piezaY):
2     si piezaX < 0 -> colision
3
4     calcular anchoPieza (columna mas a la derecha con bit activo)
5     si piezaX + anchoPieza > ancho -> colision
6
7     para fila = 0 hasta 2:
8         si pieza[fila] == 0 -> saltar fila
9
10        si piezaY + fila >= alto -> colision
11
12        filaPieza = pieza[fila] expandido a uint32_t << 24
13        filaPieza = filaPieza >> piezaX
14
15        para b = 0 hasta bytesPerFila-1:
16            desplazamiento = 24 - (b * 8)

```

```

17         bytesPieza = (filaPieza >> desplazamiento) & 0xFF
18         si (bytesPieza AND tablero[piezaY+fila][b]) != 0
19             -> colision
20
21     retornar sin colision

```

Listing 5: Pseudocódigo – Detección de colisiones

```

1  int hayColision(uint8_t** tablero, int alto, int ancho,
2      uint8_t* pieza, int piezaX, int piezaY){
3      if(piezaX < 0) return 1;
4      int anchoPieza = 0;
5      for(int fila = 0; fila < 3; fila++){
6          for(int col = 2; col >= 0; col--){
7              if((pieza[fila] >> (7 - col)) & 1){
8                  if(col + 1 > anchoPieza)
9                      anchoPieza = col + 1;
10                 break;
11             }
12         }
13     }
14     if(piezaX + anchoPieza > ancho) return 1;
15     for(int fila = 0; fila < 3; fila++){
16         if(pieza[fila] == 0) continue;
17         if(piezaY + fila >= alto) return 1;
18         uint32_t filaPieza = (uint32_t)pieza[fila] << 24;
19         filaPieza = filaPieza >> piezaX;
20         int bytesFilas = ancho / 8;
21         for(int b = 0; b < bytesFilas; b++){
22             int desplazamiento = 24 - (b * 8);
23             int bytesPieza = (filaPieza >> desplazamiento) & 0xFF;
24             if((bytesPieza & tablero[piezaY + fila][b]) != 0)
25                 return 1;
26         }
27     }
28     return 0;
29 }

```

Listing 6: Implementación – Detección de colisiones

3.4. Rotación de piezas

La rotación se calcula algorítmicamente usando la fórmula matemática de rotación 90° horario en una matriz 3×3: $nuevaFila = col$, $nuevaCol = 2 - fila$. Para cada bit activo de la pieza original se lee su posición con `getBit`, que usa `(pieza[fila] >> (7-col)) & 1` para aislar el bit deseado. Luego se calcula la nueva posición y se escribe con `setBit`, que usa `pieza[fila] |= (0x80 >> col)` para activar el bit sin modificar los demás. La pieza O es un caso especial, ya que retorna una copia sin rotar porque visualmente es igual en

todas las rotaciones. Antes de aplicar la rotación se verifica colisión con la pieza rotada, si hay colisión, se descarta.

```

1 rotacion(pieza, tipo):
2     si tipo == 0 (pieza 0):
3         retornar copia exacta sin rotar
4
5     rotada = nuevo arreglo de 3 bytes, inicializado en 0
6     para fila = 0 hasta 2:
7         para col = 0 hasta 2:
8             si getBit(pieza, fila, col) == 1:
9                 nuevaFila = col
10                nuevaCol = 2 - fila
11                setBit(rotada, nuevaFila, nuevaCol)
12     retornar rotada
13
14 getBit(pieza, fila, col):
15     retornar (pieza[fila] >> (7 - col)) & 1
16
17 setBit(pieza, fila, col):
18     pieza[fila] |= (0x80 >> col)

```

Listing 7: Pseudocódigo – Rotación 90 grados horario

```

1 int getBit(uint8_t* pieza, int fila, int col) {
2     return (pieza[fila] >> (7 - col)) & 1;
3 }
4
5 void setBit(uint8_t* pieza, int fila, int col) {
6     pieza[fila] |= (0x80 >> col);
7 }
8
9 uint8_t* rotacion(uint8_t* pieza, int tipo) {
10     if(tipo == 0) {
11         uint8_t* copia = new uint8_t[3]();
12         copia[0] = pieza[0];
13         copia[1] = pieza[1];
14         copia[2] = pieza[2];
15         return copia;
16     }
17     uint8_t* rotada = new uint8_t[3]();
18     for(int fila = 0; fila < 3; fila++) {
19         for(int col = 0; col < 3; col++) {
20             if(getBit(pieza, fila, col)) {
21                 int nuevaFila = col;
22                 int nuevaCol = 2 - fila;
23                 setBit(rotada, nuevaFila, nuevaCol);
24             }
25         }
26     }
27 }

```

```
26     }
27     return rotada;
28 }
```

Listing 8: Implementación – Rotación de piezas

3.5. Fijación de piezas

Para fijar las piezas al tablero, volvemos a aplicar la lógica de expansión de bytes con el fin de evitar la pérdida de información. La diferencia, es que en lugar de AND se usa OR, ya que este activa los bits de la pieza en el tablero sin modificar los bits que ya estaban ocupados. Después de fijar la pieza se llama a limpiarFilas para eliminar filas completas(en caso de que las haya), se libera la memoria de la pieza fijada y se genera una nueva pieza aleatoria.

```
1 fijarPieza(tablero, ancho, pieza, piezaX, piezaY):
2     para fila = 0 hasta 2:
3         si pieza[fila] == 0 -> saltar fila
4
5         filaPieza = pieza[fila] expandido a uint32_t << 24
6         filaPieza = filaPieza >> piezaX
7
8         para b = 0 hasta bytesPerFila-1:
9             desplazamiento = 24 - (b * 8)
10            bytesPieza = (filaPieza >> desplazamiento) & 0xFF
11            tablero[piezaY+fila][b] = tablero[piezaY+fila][b] OR
                bytesPieza
```

Listing 9: Pseudocódigo – Fijación de pieza al tablero

```
1 void fijarPieza(uint8_t** tablero, int ancho,
2               uint8_t* pieza, int piezaX, int piezaY){
3     int bytesFilas = ancho / 8;
4     for(int fila = 0; fila < 3; fila++){
5         if(pieza[fila] == 0) continue;
6         uint32_t filaPieza = (uint32_t)pieza[fila] << 24;
7         filaPieza = filaPieza >> piezaX;
8         for(int b = 0; b < bytesFilas; b++){
9             int desplazamiento = 24 - (b * 8);
10            uint8_t bytesPieza = (filaPieza >> desplazamiento) & 0
                xFF;
11            tablero[piezaY + fila][b] |= bytesPieza;
12        }
13    }
14 }
```

Listing 10: Implementación – Fijación de pieza al tablero

3.6. Limpieza de filas

Una fila está llena cuando todos sus bytes son 0xFF , es decir, todos sus bits están en 1. El algoritmo recorre el tablero de abajo hacia arriba para evitar saltarse filas llenas continuas, al encontrar una fila llena se libera su memoria con `delete[]`, se mueven los punteros de todas las filas superiores una posición hacia abajo, y se crea una nueva fila vacía en la cima. Mover punteros en lugar de contenido hace esta operación muy eficiente. El `i++` después de eliminar compensa el `i--` del `for` para que la misma posición se revise de nuevo.

```

1 limpiarFilas(tablero, alto, ancho):
2     eliminadas = 0
3     para i = alto-1 hasta 0 (de abajo hacia arriba):
4         si filaLlena(tablero[i]):
5             liberar tablero[i]
6             eliminadas++
7             para j = i hasta 1:
8                 tablero[j] = tablero[j-1]    <- mover punteros
9             tablero[0] = nueva fila vacia
10            i++    <- revisar misma posicion de nuevo
11    retornar eliminadas
12
13 filaLlena(fila, bytesPerFila):
14     para b = 0 hasta bytesPerFila-1:
15         si fila[b] != 0xFF -> retornar falso
16     retornar verdadero

```

Listing 11: Pseudocódigo – Limpieza de filas llenas

```

1 int filaLlena(uint8_t* fila, int bytesFila){
2     for(int b = 0; b < bytesFila; b++){
3         if(fila[b] != 0xFF)
4             return 0;
5     }
6     return 1;
7 }
8
9 int limpiarFilas(uint8_t** tablero, int alto, int ancho){
10    int bytesFilas = ancho / 8;
11    int eliminadas = 0;
12    for(int i = alto - 1; i >= 0; i--){
13        if(filaLlena(tablero[i], bytesFilas)){
14            delete[] tablero[i];
15            eliminadas++;
16            for(int j = i; j > 0; j--){
17                tablero[j] = tablero[j-1];
18            }
19            tablero[0] = new uint8_t[bytesFilas]();
20            i++;
21        }

```

```
22     }  
23     return eliminadas;  
24 }
```

Listing 12: Implementación – Limpieza de filas

4. Problemas de desarrollo

Durante el desarrollo del proyecto, surgieron distintos obstáculos que se resumen a continuación:

- **Bucle infinito en la validación de la entrada:** Más que un error de código, fue realmente un error de configuraciones. Al realizar pruebas con las funciones de validar tablero e imprimir tablero, el programa entraba en un loop infinito, pidiendo una y otra vez la entrada de ancho y altura, sin detenerse a permitir el ingreso del dato por parte del usuario. Intentamos de diversas formas solucionarlo, cambiando la estructura del bucle e intentado limpiar el buffer del cin en caso de datos basura, para luego darnos cuenta de que faltaba la opción de correr en terminal. Una vez aplicado esto, fue posible correr el código con normalidad.
- **Frontera inferior invisible:** Durante las pruebas de los movimientos de las piezas, todo funcionó con normalidad, excepto por una cosa. Al querer bajar una pieza hasta el fondo, algunas de ellas se fijaban en la penúltima fila. Esto sucedió porque en ese momento, la variable que representaba la referencia Y para las piezas estaba dada por un número constante (3) haciendo que se tomara siempre toda la matriz aunque la última fila estuviera vacía. Una vez que cambiamos esto por la variable **fila**, el problema se solucionó.
- **Limites laterales invisibles al rotar:** Al hacer la implementación y pruebas de las rotaciones, estas colisionaban con límites laterales invisibles. Esto se debía a que para las colisiones, estábamos comparando siempre con la pieza original, en vez de la rotada, se solucionó simplemente actualizando los parámetros.
- **Tipo no se actualizaba:** Para identificar la pieza a rotar se tenía una variable llamada **tipo**, la cual al estar en 0, omitía las rotaciones por tratarse de la pieza cuadrada. Sin embargo, todas las otras fichas estaban entrando a ese caso, notamos que se debía a que la variable se actualizaba de manera local en la función encargada de generar pieza pero ese cambio solo vivía en ese ámbito. Lo solucionamos pasando el parámetro por referencia.

5. Evolución de la solución

El desarrollo del proyecto siguió un orden paralelo de implementaciones. Primero se definieron las estructuras de datos y los lineamientos generales y luego, aprovechando los módulos, cada integrante trabajó en ellos, proponiendo y complementando soluciones.

Primera etapa – Diseño de estructuras de datos

Antes de escribir cualquier función se definió cómo se representarían el tablero y las piezas en memoria. Decididos usar `uint8_t` como base para garantizar portabilidad, y se estableció el uso de bits más significativos para las columnas de las piezas. Esta etapa fue la más importante porque todas las decisiones posteriores se basaron en ella.

Segunda etapa – Implementación por módulos

Se implementaron los módulos. Gracias a su independencia, se podía trabajar en varios de forma simultánea, permitiendo avanzar con mayor rapidez. En cuanto a el orden, no fue necesario seguir uno, ya que cada módulo era independiente, solo el main tuvo que ser dejado para el final ya que este sí dependía de las funciones definidas en los otros módulos.

Cambios durante el desarrollo

Durante el proceso se tomaron varias decisiones que implicaron cambios de enfoque. Inicialmente se consideró representar las piezas en una grilla 4×4 , pero se optó por 3×3 ya que todas las piezas implementadas caben en ese espacio y reduce el uso de memoria. También se consideró guardar e invocar las 4 rotaciones de cada pieza, pero el profesor indicó que debían calcularse algorítmicamente con bits, lo que llevó a implementar la fórmula `nuevaFila = col, nuevaCol = 2 - fila`.

Consideraciones para mejoras futuras

El juego actual baja la pieza solo cuando el usuario presiona **S**. Una mejora sería agregar gravedad automática que la pieza baje sola cada cierto número de turnos. También se podría agregar un sistema de puntuación basado en la cantidad de filas eliminadas, y mostrar la siguiente pieza al usuario antes de que aparezca en el tablero.